Learn Python

Teacher Edition

Minute-by-minute session playbooks · transition scripts · predicted misconceptions · Socratic prompts

A 10-session, ~10-hour (one hour per session, + capstone) accelerated Python course for a PhD-in-Education learner with no prior coding experience.

Generated 2026-07-06 · Instructional content original

Learn Python — TEACHER Edition

Everything in the student syllabus, **plus** the instructor scaffolding: a minute-by-minute clock for each one-hour session, transition scripts (how to move between blocks without dead air), predicted misconceptions for *this* learner, Socratic prompts (DeepTutor-style — ask, don't tell), and an explicit **"if you're behind, cut this"** line per session.

The 10 sessions run in five natural pairs (types → traps, control flow → data structures, functions → recursion, exceptions → files, regex → modules/OOP). Each pair shares a through-line; **name it out loud** when you open the second session of a pair, so consecutive hours feel like one arc.

How to read this document

Each session has:

- **Covers** — the one topic this hour owns.
- **Pre-flight** — what to have open/ready before the student arrives.
- **The clock** — a 60-minute breakdown. Keep a timer visible. The numbers are targets, not law.
- **Transitions** — short scripted lines to hand off cleanly between blocks.
- **Predicted misconceptions** — where THIS learner (expert in research, novice in code) will stumble.
- **Socratic prompts** — questions to ask instead of explaining; let him derive it.
- **Cut line** — the first thing to drop if you're running over.
- **Homework** — what to assign at the close (specs live in `examples/session-NN/practice.md` → *Homework*).

Universal pacing principles (this learner is fast)

- **Talk less than you want to.** He reads fast and abstracts well. Default to "here's the rule, here's the trap, now you try."
- **Protect the practice block.** ~18 minutes of hands-on in every hour, plus the live-coding he types along with. If concept overruns, steal from your own talking, never from his typing.
- **Predict-then-run is the engine, especially the Session 2 traps.** Always have him *commit to an answer out loud* before running. The cognitive surprise is the teaching moment.
- **The warm-up is homework-powered.** Every session (except S1) opens with 5 minutes on the previous homework + the misconception log. Don't re-teach — quiz. If homework wasn't done, do H1 together as the warm-up and move on.
- **Fast finisher?** Send him to the session's **Extra practice** block, not more of your talking.
- **Don't pad.** Each clock is tight by design. If a block ends early, bank the minutes for practice.
- **Carry a running "misconceptions log"** (DeepTutor "learning memory"): note every trap he hit this session; re-surface it as the next warm-up.

SESSION 1 — Running Python, Variables & Types

Covers: REPL vs script, variables as labels, the five core types, `input()` / `print()`, f-strings, casting, reading a traceback. **Pre-flight:** terminal + VS Code open; `examples/session-01/` ready; a deliberately broken line staged to show a traceback.

The clock (60 min)

- **0:00–0:05 — Orientation.** Why Python, why this re-ordered path, how the hour runs, what homework is for. Don't oversell; he wants to start.
- **0:05–0:25 — Concept.** REPL vs script; `python3 file.py`. Variables as *labels on objects* (Connection Map #1). The five core types; `type(x)`. `input()` → always `str`. f-strings. `int()/float()/str()`; `int(3.9)` truncates.
- **0:25–0:37 — Live (he types along).** Build `greet.py`, then "years to graduation"; trigger and *read* one traceback together (last line first).
- **0:37–0:55 — Practice.** `examples/session-01/practice.md` (In class): GPA reporter, age bucket, the string-concatenation trap. Ahead? → Extra practice (f-string drill).
- **0:55–1:00 — Recap + quiz + homework.** Traps recap: `input()` → string; `int(3.9)` truncates; `print(a, b)` vs `print(a + b)`. Quiz S1. Assign homework.

Transitions

- Concept→live: *"Enough theory — watch me make these mistakes so you don't have to."*
- Live→practice: *"Your turn. Same moves, your fingers."*
- Close: *"You now know the five types. Next hour is the whole reason this course exists: the ways Python lets those types surprise you."*

Predicted misconceptions (this learner)

- Expects `input()` to give a number → `"5" + "3" == "53"`.
- Thinks `=` asserts equality (math habit). Reinforce label-on-object now; it pays off in S2 and in S4's aliasing.
- Expects `int(3.9)` to round.

Socratic prompts

- "What type do you think `input()` hands back? How could we check?" (→ `type(...)`)
- "Why did `'5' + '3'` not crash, and not give 8 either?"
- "The traceback is nine lines. Which one pays rent?"

Cut line: drop the years-to-graduation demo; never cut the traceback reading or the practice.
Homework: unit converter · traceback drill · type-prediction table.

SESSION 2 — The Dynamic-Typing Traps *(the most important hour)*

Covers: `==` vs `is`, `bool ⊂ int`, float precision, cross-type comparison, truthiness, `isinstance` vs `type`. **Pre-flight:** `examples/session-02/`; the trap gauntlet open (site page or notebook); `cheatsheets/traps-and-gotchas.md` ready to hand over at the end.

The clock (60 min)

- **0:00–0:05 — Warm-up.** S1 homework debrief: one traceback from his drill, one row of the type table. Log anything shaky.
- **0:05–0:22 — Concept (compressed — the demos carry it).** This is the heart of the course; tell him so. `==` value vs `is` identity (Connection Map #3), `is None`; `bool ⊂ int` (`True == 1`, `sum(flags)`, Connection Map #4); floats are binary → `0.1 + 0.2` → `math.isclose` (Connection Map #5); `5 == "5"` False but `5 > "5"` raises; `isinstance` vs `type`; truthiness.
- **0:22–0:40 — The gauntlet (predict-then-run).** ~18 one-line traps. He predicts every line *out loud* before you run it. This is live and practice fused — his hands on the keyboard, your questions in the air.
- **0:40–0:55 — Practice.** `examples/session-02/practice.md` (In class): the predict-the-output gauntlet on paper, then `clean_score()` handling int/float/str and rejecting `bool`.
- **0:55–1:00 — Recap + quiz + homework.** *"These traps are 80% of week-one bugs."* Hand over the trap cheat sheet as his permanent reference. Quiz S2. Assign homework (the trap journal seeds his bug log).

Transitions

- Warm-up→concept: *"You now know the five types. Here's the catch: Python lets them mix in ways that surprise everyone. Cover the right column and predict each line."*
- Concept→gauntlet: *"Rule time is over. Eighteen traps. Say your answer out loud, then we run it."*
- Close: *"Every one of these you predicted wrong goes in your journal tonight — that's the homework, and it becomes your personal cheat sheet."*

Predicted misconceptions

- Assumes `is` is a stylistic `==` — nail it with the mutable-list demo.
- Trusts `==` on floats out of stats habit ("I round anyway") — stress the *cause* is binary storage, not data.
- Over-generalizes the `TypeError` and thinks `5 == "5"` errors too. It returns `False` — show it.
- Thinks `"0"` is falsy.

Socratic prompts

- "Two students, same 3.7 GPA. `==`? `is`? Why?"
- "If `0.1 + 0.2` isn't `0.3`, is the bug in the data or in the computer? How would you test 'close enough'?"
- "`5 == '5'` is False, fine. So why does `5 > '5'` *crash*?"

- "`sum([True, False, True])` — why is that even legal? And when is it useful?" (→ counting flags)

Cut line: drop the small-int-cache and `nan` wrinkles; **never** cut `==` / `is`, floats, `isinstance`, or `clean_score()`. **Homework:** trap journal · `approx_equal` · `is_missing`.

SESSION 3 — Control Flow: Conditionals & Loops

Covers: `if/elif/else`, chained comparisons, `and` / `or` / `not`, `for` / `while`, `range`, `break` / `continue`, `enumerate` / `zip`, the validation loop. **Pre-flight:** `examples/session-03/`; a messy nested-`if` staged for the refactor; `Ctrl+C` ready for the infinite-loop demo.

The clock (60 min)

- **0:00–0:05 — Warm-up.** S2 homework debrief: two entries from his trap journal; quiz `approx_equal(0.1+0.2, 0.3)`.
- **0:05–0:25 — Concept.** `if/elif/else`; **chained comparisons** (`90 <= x < 100` — reads like math); `and/or/not`, short-circuit, `and` / `or` return an *operand*; `while` (+ infinite-loop demo); `for ... in`; `range` (off-by-one!); `break` / `continue`; the `while True: … break` validation pattern; **`enumerate` / `zip`** as the antidote to `range(len(...))`.
- **0:25–0:37 — Live.** A Likert→label classifier (chained comparison + early `return`); sum/average a roster two ways (index vs `enumerate` / `zip`); a robust "ask until valid" loop.
- **0:37–0:55 — Practice.** `examples/session-03/practice.md` (In class): grade-band classifier — **test every boundary** (89.999/90/90.001) — logic drill, `zip` pass/fail, validation loop, mutate-while-iterating trap.
- **0:55–1:00 — Recap + quiz + homework.** `if x == True` → `if x`; `range(1,5)` excludes 5; don't mutate a list you're looping. Quiz S3.

Transitions

- Concept→live: *"Watch me turn an ugly five-level `if` into three readable lines — then loop a whole roster."*
- Live→practice: *"Boundaries are where the bugs live. Make 89.999, 90, and 90.001 all behave."*
- Close: *"You can branch and repeat. Next hour: the containers worth looping over — and a list of dicts is secretly your dataset."*

Predicted misconceptions

- Writes `if score >= 90 and score < 100` — show chaining `90 <= score < 100`.
- Boundary errors (`>=` vs `>`) at cutoffs — make him test 89.999 / 90 / 90.001.
- `range(len(x))` index habit (from R/SPSS vectorized thinking) — push `enumerate` / `zip`.
- Removes items from a list *while iterating it* → demo the bug, then build a new list.

Socratic prompts

- "`x = 5 and 0` — what's `x`? Why isn't it `True` / `False`?"
- "You need both the position and the value. What's cleaner than indexing?"
- "How many numbers does `range(1, 5)` produce? Prove it."

Cut line: drop `match/case` and `for/else`; keep chained comparisons, `enumerate` / `zip`, and the validation loop. **Homework:** attendance labeler · number-guessing game · leap-year checker.

SESSION 4 — Data Structures

Covers: `list` / `tuple` / `dict` / `set`, slicing, nesting (list of dicts = dataset), comprehensions, `sorted(key=…)`, aliasing. **Pre-flight:** `examples/session-04/`; a "list of dicts = tidy dataset" diagram (Connection Map #6); the aliasing demo staged.

The clock (60 min)

- **0:00–0:05 — Warm-up.** S3 homework debrief: `label(79.9)`? `is_leap(1900)`? One boundary he got wrong, re-run.
- **0:05–0:25 — Concept.** `list` (mutable)/ `tuple` (immutable)/ `dict` (key→value)/ `set` (unique); indexing & **slicing**; nesting → **a list of dicts is a dataset** (rows as dicts, keys as your variables); list/dict comprehensions; `sorted(key=lambda …)`.
- **0:25–0:37 — Live.** Build a roster as a list of dicts, sort by score, dedupe answers with a `set`, rewrite a loop as a comprehension — then the **aliasing** bug (`b = a`, mutate `a`, watch `b`) and its fix.
- **0:37–0:55 — Practice.** `examples/session-04/practice.md` (In class): rank, `{name: score}` comprehension, group pass/fail, dedupe, reproduce-then-fix aliasing.
- **0:55–1:00 — Recap + quiz + homework.** `=` aliases; `.sort()` returns `None`; `.get()` beats `KeyError`; `[[0]*3]*3` shares rows. Quiz S4.

Transitions

- Open (the pair's thread): *"You can loop over data now. This hour: the containers worth looping — and a list of dicts is just a tidy dataset."*
- Live→aliasing: *"One last thing that bites everyone — assignment doesn't copy. Watch `b` change when I never touched it."*
- Close: *"Data's handled. Next hour we stop repeating ourselves: functions."*

Predicted misconceptions

- **Aliasing** is the big one — expects `b = a` to copy. Show `a.append(...)` changing `b`; tie to S1 "label on object" and S2 `is`.
- Expects `xs.sort()` to return the sorted list (it returns `None`).
- Reaches for a list when uniqueness is the requirement (→ `set`).

Socratic prompts

- "`b = a; a.append(99)` — what's in `b`? Why? (Labels, not boxes.)"
- "Survey gave duplicate free-text answers. What structure removes duplicates for free?"
- "Each dict is a row, each key a column. What's your unit of analysis here?"

Cut line: drop deep-copy/ `[[0]*3]*3` (homework covers the grid); keep comprehensions and aliasing basics. **Homework:** gradebook dict drill · frequency counter · fix-the-grid.

SESSION 5 — Functions, Scope & Reusability

Covers: `def`, parameters (positional/keyword/default, `*args` / `**kwargs`), `return` vs `print`, LEGB scope, docstrings, type hints, the mutable-default bug. **Pre-flight:** `examples/session-05/`; the **mutable-default-arg** demo staged (the headline); S4's inline code ready to refactor.

The clock (60 min)

- **0:00–0:05 — Warm-up.** S4 homework debrief: the grid bug — *why* did every row change? (Aliasing, again. It should be automatic now.)
- **0:05–0:25 — Concept.** `def`; parameters (positional/keyword/default, `*args` / `**kwargs`); `return` vs `print`; scope (LEGB), `global` and why to avoid it; docstrings; **type hints** ("not enforced; `mypy` checks them").
- **0:25–0:37 — Live.** Refactor S4 inline code into `class_average` / `letter_grade`; then the **mutable-default bug** live (`def add(x, bag=[])` accumulating across calls) → fix with `bag=None`.
- **0:37–0:55 — Practice.** `examples/session-05/practice.md` (In class): grade-functions library with hints/docstrings; reproduce-then-fix the mutable default; `summary(*scores)`.
- **0:55–1:00 — Recap + quiz + homework.** Mutable default → `None`; forgot `return` → `None`; assigning a global → `UnboundLocalError`; hints aren't enforced. Quiz S5.

Transitions

- Concept→live: *"This next bug has burned every Python programmer once. Watch the list keep growing across calls."*
- Live→practice: *"Now you cause the bug on purpose, then cure it. Bugs you've caused are bugs you recognize."*
- Close: *"A function can call other functions. Next hour, the mind-bender: it can call itself."*

Predicted misconceptions

- Won't believe the default list persists until shown twice; then explain defaults evaluate *once, at definition* — and note it's the S4 aliasing story again.
- Thinks a function that `print`s has "returned" the value → show `x = show(...)` is `None`.
- Expects type hints to enforce types → show `add("a","b")` still runs.

Socratic prompts

- "I called `add(1)` three times and the list keeps growing. When does `bag=[]` actually run?"
- "`print` vs `return` — which lets the *next* function use the result?"
- "Where does Python look for a name first? (L-E-G-B — walk it.)"

Cut line: drop decorators/closures and keyword-only args (Extra practice has them); **never** cut the mutable-default demo. **Homework:** tiny stats library · `mean_ignoring_none` · scope prediction.

SESSION 6 — Recursion & Recursive Thinking

Covers: base case + recursive case, tracing the call stack, recursion vs iteration, nested data, `RecursionError`. **Pre-flight:** `examples/session-06/`; a nested-JSON-shaped dict staged for `deep_sum`; the `runaway` overflow + `sys.getrecursionlimit()` queued.

The clock (60 min)

- **0:00–0:05 — Warm-up.** S5 homework debrief: `mean_ignoring_none(None)`? The scope drill — what error, and why?
- **0:05–0:25 — Concept.** The two parts: a **base case** and a **recursive case** that moves toward it. Trace `orderings(3)` as a stack that builds then unwinds; recursion vs iteration; the cost (each pending call is a frame; **no tail-call optimization**, limit ≈ 1000). The prereq-chain example (`prereqs_deep`).
- **0:25–0:37 — Live.** `countdown` / `factorial`, then `deep_sum` over nested data (the payoff a single loop can't reach), then a `RecursionError` read together. Mention `@functools.cache`.
- **0:37–0:55 — Practice.** `examples/session-06/practice.md` (In class): recursive `total`, `reverse`, `flatten`, `depth`, and the two trap-fixes (missing base case; forgetting to `return`).
- **0:55–1:00 — Recap + quiz + homework.** Reachable base case or overflow; `return` the recursive call; loops for flat work. Quiz S6.

Transitions

- Open (the pair's thread): *"A function can call other functions. The mind-bender: it can call* itself*. That's exactly the tool for data defined in terms of itself — nested data."*
- Live→practice: *"Say the base case out loud before you write the function — that's where the bugs hide."*
- Close: *"You handle clean data beautifully. Next hour: what to do when the data fights back."*

Predicted misconceptions

- Writes the recursive case but forgets to `return` it → silent `None`.
- From a vectorized stats background, may not see when recursion beats a loop → the nested-data demo is the "aha".
- May assume recursion is a free swap for loops → show the limit.

Socratic prompts

- "What's the smallest input where the answer is obvious without recursing? That's your base case."
- "Your data is a list that can contain lists. What kind of function matches a thing defined in terms of itself?"
- "Each pending call sits on a stack. What happens at call #1001?"

Cut line: drop `depth`/string-reverse tasks and `@cache`; **never** cut `deep_sum` on nested data. **Homework:** `sum_digits` · `deep_count` · loop-vs-recursion paragraph.

SESSION 7 — Exceptions & Defensive Code

Covers: `try` / `except` / `else` / `finally`, exception types, `raise`, EAFP vs LBYL, `assert`, a first `pytest` test. **Pre-flight:** `examples/session-07/`; a dirty list ("N/A", "", "7" on a 1–5 scale) staged; `pytest` installed and verified.

The clock (60 min)

- **0:00–0:05 — Warm-up.** S6 homework debrief: `sum_digits(4823)` trace; why did `deep_count` need the bool check? (S2 pays rent again.)
- **0:05–0:25 — Concept.** Errors vs exceptions; `try/except/else/finally`; common types (`ValueError`, `KeyError`, `FileNotFoundError`); `raise ValueError(...)`; **EAFP** vs LBYL; `assert` (a developer check, can be disabled — not input validation).
- **0:25–0:37 — Live.** Harden `safe_int()` / `clean_likert()` against blanks/"N/A"/out-of-range; a first `pytest.raises` test, run it, watch it pass.
- **0:37–0:55 — Practice.** `examples/session-07/practice.md` (In class): validate a raw response list into clean values + a rejection log; add a `raise`; one pytest test.
- **0:55–1:00 — Recap + quiz + homework.** Never bare `except:`; don't swallow errors; `assert` ≠ validation. Quiz S7.

Transitions

- Open: *"Your real survey data WILL have 'N/A' in a numeric column. This hour we write code that shrugs it off."*
- Live→practice: *"Here are eight dirty values. I want a clean list AND a rejection log — you never silently discard data."*
- Close: *"You can survive one bad value. Real data is a* file full *of them — next hour we open a real CSV and clean it with exactly these moves."*

Predicted misconceptions

- Reaches for `if/else` (LBYL) everywhere; introduce EAFP as the Pythonic default, but be honest both are valid.
- Writes bare `except:` — show why `except ValueError:` is safer.
- Confuses `raise` with `return`, and `assert` with input validation.

Socratic prompts

- "User typed 'seven' instead of 7. Catch it *before* (`if`) or *after* (`try`)? Trade-offs?"
- "Why is a bare `except:` dangerous? What might you swallow?"
- "You rejected four values. Should the program say so? Where's the audit trail?"

Cut line: drop the custom exception subclass (Extra practice has it) and shrink pytest to one test; keep `try/except` validation. **Homework:** `ask_int` · three pytest cases · error triage.

SESSION 8 — Files, Libraries & Research Data

Covers: `open` / `with`, file modes, CSV via `DictReader` / `DictWriter`, `json`, the researcher's stdlib, the pandas teaser. **Pre-flight:** `examples/session-08/` with `students.csv` + `survey.csv`; confirm `pip` works and have `pandas` importable for the teaser.

The clock (60 min)

- **0:00–0:05 — Warm-up.** S7 homework debrief: one error-triage item; `ask_int` — what happens on `" 42 "`, and why does EAFP win there?
- **0:05–0:25 — Concept.** `open` / `with` (auto-close); modes (`r/w/a`; `w` **overwrites!**); **CSV** via `csv.DictReader` / `DictWriter` (rows as dicts → ties to S4); `json`; researcher stdlib (`statistics`, `datetime`, `pathlib`); `pip install`.
- **0:25–0:37 — Live.** Read `students.csv` → list of dicts, class mean with `statistics.mean`, write a summary CSV; then the **`pandas` teaser** (*"same thing in three lines — that's your next course"*).
- **0:37–0:55 — Practice.** `examples/session-08/practice.md` (In class): per-item survey means skipping dirty values, write `survey_summary.csv`, mean by major — S7's exception skills do the cleaning.
- **0:55–1:00 — Recap + quiz + homework.** Bare `"w"` destroys data; `newline=""`; files exhaust after one read; CSV values are strings. Quiz S8.

Transitions

- Open (the pair's thread): *"You can now survive one bad value. Real data is a file full of them — let's open a real CSV."*
- Teaser framing: *"Everything you just did by hand, pandas does in three lines — your next course, not today's. Now you know what it's doing underneath."*
- Close: *"Numbers are clean. Next hour: the messiest data of all — text."*

Predicted misconceptions

- Opens with `"w"` to read and wonders why the file is empty; stress mode meanings.
- Expects to iterate a file object twice without re-opening; show the exhausted cursor.
- Forgets CSV values are strings (`"91" + 1` → `TypeError` — S2 again).

Socratic prompts

- "Why does `with` matter even if your program crashes? What does it guarantee?"
- "The score column came back as `'91'`. What must happen before math?"
- "Your summary has `True` in it. What will JSON write there?" (→ `true` — JSON is its own language)

Cut line: drop the `pandas` / `json` teaser; keep `csv.DictReader/Writer` and the dirty-value cleaning. **Homework:** attendance-report pipeline · JSON round-trip · your own CSV.

SESSION 9 — Regular Expressions & Text Cleaning

Covers: raw strings, the survival tokens, `search` / `fullmatch` / `findall` / `sub`, capture groups, when NOT to use regex. **Pre-flight:** `examples/session-09/`; messy free-text, emails, and "Last, First" names staged; regex101.com open (flavor: Python).

The clock (60 min)

- **0:00–0:05 — Warm-up.** S8 homework debrief: what did JSON do to `True`? What dirty value did his own CSV throw, and how did he handle it?
- **0:05–0:25 — Concept.** Why regex for a researcher (validate/extract/clean/qualitative-coding, Connection Map #9). **Raw strings** `r"..."`. Survival tokens (`. \d \w \s + * ? {m,n} ^ $ [] () |`); `.` matches *any* char (use `\.`). The four functions: `search` / `fullmatch` / `findall` / `sub`; capture groups.
- **0:25–0:37 — Live.** Validate an email (`fullmatch`), extract dept+number with groups, collapse whitespace with `sub`, count `#hashtags` with `findall` — and one case where `.split()` beats regex.
- **0:37–0:55 — Practice.** `examples/session-09/practice.md` (In class): email validator, extract codes, hashtag count, the `"Last, First"` flip, the judgment call.
- **0:55–1:00 — Recap + quiz + homework.** `r"..."` always; `\.` for a literal dot; guard `None` before `.group()`. Quiz S9.

Transitions

- Concept→live: *"Four functions cover almost everything: search, fullmatch, findall, sub. Every pattern is a raw string, no exceptions."*
- Live→practice: *"Regex matches form, not meaning. You decide what the pattern should be — then make Python agree."*
- Close: *"You can now clean any string. Last hour of new material: organizing your code so it's reusable."*

Predicted misconceptions

- Forgets raw strings → backslash chaos. Always `r"..."`.
- Calls `.group()` on a `None` result → teach the `if m:` guard (same shape as `5 > '5'` from S2).
- Expects regex to understand meaning (qualitative-coding hopes) — it matches *form*.

Socratic prompts

- "You want every response mentioning a theme. Is that a *form* match (regex) or a *meaning* match (human coding)? What can regex actually catch?"
- "Why does `re.search(...).group()` crash when there's no match? (Same shape as `5 > '5'` weeks ago.)"
- "Would you regex-split `'a,b,c'`? What's the simpler tool?"

Cut line: drop named groups / `re.VERBOSE` (Extra practice has named groups); keep validate + extract + `sub`. **Homework:** pattern drill · messy-name cleanup · domain harvest.

SESSION 10 — Modules, OOP & the Pythonic Toolkit

Covers: modules & `import`, the `__main__` guard, a class with `__init__` / `self` / `__str__`, a validating `@property`, inheritance, generators, `map` / `filter`, walrus. **Pre-flight:** `examples/session-10/`; `grades.py` ready to import; the `Student` class and the generator-exhaustion demo staged.

The clock (60 min)

- **0:00–0:05 — Warm-up.** S9 homework debrief: his three patterns — run each against one *invalid* example; did `fullmatch` catch it?
- **0:05–0:25 — Concept.** Modules: move grade functions to `grades.py`, `import`, the `if __name__ == "__main__":` guard. OOP: a small `Student` class — `__init__`, `self`, a method, `__str__`, a validating `@property` setter (Connection Map #10), brief inheritance with `super()`.
- **0:25–0:37 — Live.** Build the validating `Student`; `ana.gpa = 5.0` raises — the object defends its own integrity. Then the toolkit tour-by-doing: comprehension, `map` / `filter`, a generator + its one-pass exhaustion, the walrus `:=`.
- **0:37–0:55 — Practice.** `examples/session-10/practice.md` (In class): import from `grades.py`, the validating `Student`, `GradStudent(super())`, toolkit drills.
- **0:55–1:00 — Recap + quiz + homework + course wrap.** `self` is just "this instance"; generators exhaust; don't class-ify what a dict does. Quiz S10. Frame the capstone: *"Next time, you drive."*

Transitions

- Open (the pair's thread): *"You can now clean any string. Last step: organize your code so it's reusable — functions into modules, then data-plus-behavior into a class."*
- Modules→OOP: *"Functions in a file is reuse. A class bundles the data* and *the rules that guard it."*
- Close: *"That's the toolbox — every tool in it earned its place. The capstone is where you prove it's yours."*

Predicted misconceptions

- `self` looks magical — it's just "this particular instance."
- Treats a generator like a list (iterates once) → show the second pass is empty.
- Reaches for a class when a function or dict would do — name when OOP earns its keep.

Socratic prompts

- "Your operational definition of 'student' — what attributes and rules belong to it? That's your class."
- "A million rows won't fit in memory. What does `yield` give you that a list doesn't?"
- "Why does the demo code in `grades.py` NOT run when you import it?"

Cut line: drop `@dataclass` (Extra practice) and compress the toolkit tour to comprehensions + generators; keep modules and the validating class. **Homework:** `Student` with computed GPA · `Cohort` class · Pythonic rewrite.

SESSION 11 (Optional) — Capstone

Role shift: you stop teaching and start *coaching*. He drives; you ask questions and unblock. Plan ~2 hours.

- **0:00–0:15 — Brief & plan.** He restates the goal and sketches the steps aloud (pseudocode). You only check the plan is sound.
- **0:15–1:05 — Build.** He codes the Gradebook & Survey Analyzer (`assessments/capstone-project.md`). Intervene only when stuck >3 min; prefer a question over an answer.
- **1:05–1:13 — Break.**
- **1:13–1:45 — Build, continued.** Finish the report CSV, then one stretch goal (a `Student` class, a regex validation, or the recursive nested-data total).
- **1:45–1:55 — Review.** Walk his code for the course's traps (identity, aliasing, mutable defaults, bare `except:`). Praise readability.
- **1:55–2:00 — Debrief & next steps.** Point to pandas/visualization as the genuine next course.

Coaching prompts: "What's your data structure?" · "What happens if that cell is blank?" · "Is that comparing value or identity?" · "Could that be one comprehension?"

Instructor's running checklist (use across all sessions)

- [ ] Timer visible; the practice block got its full ~18 minutes.
- [ ] Warm-up pulled from the homework and the misconceptions log — not improvised.
- [ ] Pair through-lines named out loud (S2, S4, S6, S8, S10 openings).
- [ ] Every trap demoed via **predict-then-run**, not narration.
- [ ] Each new concept hooked to the **Connection Map** before syntax.
- [ ] Student typed everything himself; you talked less than half the session.
- [ ] End-of-session quiz given; homework assigned by name; capstone kept in view as the destination.

Appendix A — Master Course Outline

Master Course Outline — Learn Python

> Single source of truth. The student and teacher syllabi both derive from this. **10 core one-hour sessions** + 1 optional capstone session (~10 hours core, ~12 with the capstone). Every session also assigns **homework (~30–45 min)** that does *not* count toward class time.

How the topics are sequenced

A typical intro-Python course teaches roughly: functions/variables → conditionals → loops → exceptions → libraries → unit tests → file I/O → regular expressions → OOP → assorted "power-tools."

We **re-sequence** for a fast adult learner, and we front-load the dynamic-typing traps — they're the whole reason this course exists. The 10 sessions run in five natural pairs, each pair sharing a through-line (name it when you cross between them):

1. **Types (S1) → the type traps (S2).** You can't reason about the traps until you know the types — so they lead, back to back, on days one and two.
2. **Control flow (S3) → data structures (S4).** Loops are most useful over the containers worth looping; a list of dicts is just a dataset.
3. **Functions (S5) → recursion (S6).** Recursion is a function that calls itself, so it lands while function mechanics are fresh — and nested data (from S4) is the payoff.
4. **Exceptions (S7) → files & research data (S8).** Surviving one dirty value scales straight into cleaning a whole CSV of them.
5. **Regex (S9) → modules & OOP (S10).** The two "finishing" skills: clean any text, then organize code into modules and a small class.

The power-tools (comprehensions, generators, `*args`, type hints, the walrus) are folded into the sessions where they naturally belong, not saved for the end.

Design rules (from the e-learning pipeline)

- **Every session = one hour = one topic**, run **warm-up** → **concept** → **live example** → **practice** → **recap + quiz**, with practice protected (~25 min hands-on in every hour).
- Every session assigns **homework** (in `examples/session-NN/practice.md` → *Homework*): ~30–45 min outside class, solutions included, reviewed in the next session's warm-up. Each practice file also carries an **Extra practice** block for fast finishes — in-class stretch material, not extra talking.
- Every session has 2–3 learning objectives; every objective is testable in that session's quiz.
- Every abstract idea ships with a runnable, education-flavored example.
- Difficulty rises monotonically; nothing is used before it's introduced (except clearly-flagged teasers).

Session 1 — Running Python, Variables & Types

Objectives

1. Run Python interactively (REPL) and as a `.py` script; read a traceback without panic (last line first).
2. Use the core types (`int`, `float`, `str`, `bool`, `None`); check them with `type()`.
3. Do I/O with `input()` / `print()`, f-strings, and `int()` / `float()` / `str()` casting — and remember `input()` is *always* a `str`.

Trap focus: `input()` returns text (`"5" + "3"` is `"53"`); `int(3.9)` truncates; `print(a, b)` vs `print(a + b)`. **Homework:** unit converter; traceback drill; type-prediction table.

Session 2 — The Dynamic-Typing Traps *(the core of the course)*

Objectives

1. Distinguish **value equality (`==`)** from **identity (`is`)**; keep `is` for `None`.
2. Predict `bool ⊂ int` (`True == 1`, summing flags), int/float mixing, and **float precision** (`0.1 + 0.2`); compare floats with `math.isclose`.
3. Handle cross-type comparison (`5 == "5"` is `False`, `5 > "5"` raises); check types with `isinstance()` vs `type()`; reason about truthiness.

Why it leads: front-loading the traps — right after the types they concern — means every later session can assume the fluency. **Trap focus:** the ~18-trap predict-then-run gauntlet *is* the session. **Homework:** trap journal (5 traps in your own words); `approx_equal`; `is_missing`.

Session 3 — Control Flow: Conditionals & Loops

Objectives

1. Write `if` / `elif` / `else` with comparison & logical operators and **chained comparisons**; use `and` / `or` / `not` correctly (short-circuit, operand-return); avoid `if x == True`.
2. Write `for` / `while` loops; control them with `break` / `continue`; mind the `range` off-by-one; run the `while True:` validation loop.
3. Iterate Pythonically with `enumerate` / `zip` instead of `range(len(...))`.

Trap focus: `if x == True`; `range(1,5)` excludes 5; mutating a list while iterating it; `range(len(...))`. **Homework:** attendance labeler (boundary testing); number-guessing game; leap-year checker.

Session 4 — Data Structures

Objectives

1. Choose between `list` / `tuple` / `dict` / `set`; index, slice, and nest them — a **list of dicts is a dataset**.
2. Build list/dict **comprehensions**; sort with `sorted(key=…)`.
3. Reason about **mutability & aliasing** (copy vs reference, `[[0]*3]*3` shared rows).

Trap focus: aliasing (`b = a` shares the list); `.sort()` returns `None`; `dict.get()` vs `KeyError`; the shared-row grid. **Homework:** gradebook dict drill; frequency counter; fix-the-grid.

Session 5 — Functions, Scope & Reusability

Objectives

1. Define functions with positional, keyword, default, `*args`, and `**kwargs` parameters; know `return` vs `print`.
2. Explain scope (LEGB) and dodge the `UnboundLocalError` / `global` trap.
3. Document with docstrings and **type hints** (and know hints aren't enforced).

Trap focus: the **mutable default argument** bug; forgetting to `return`; assigning to a global inside a function. **Homework:** a tiny stats library; `mean_ignoring_none(*values)`; scope-prediction drill.

Session 6 — Recursion & Recursive Thinking

Objectives

1. Write a recursive function with a correct **base case** and **recursive case**; trace the **call stack**.
2. Convert between recursion and iteration; know recursion's cost (`RecursionError`, no tail-call optimization, limit ≈ 1000).
3. Apply recursion to **naturally nested data** (nested lists/dicts/JSON) where one loop is awkward.

Trap focus: an unreachable base case → stack overflow; forgetting to `return` the recursive call.
Homework: `sum_digits`; `deep_count` on a nested gradebook; loop-vs-recursion paragraph.

Session 7 — Exceptions & Defensive Code

Objectives

1. Handle errors with `try` / `except` / `else` / `finally`; `raise` deliberately; validate messy human/research input the EAFP way.
2. Know the common exception types on sight (`ValueError`, `TypeError`, `KeyError`, `FileNotFoundError`, …).
3. Use `assert` for developer checks (never input validation) and write a first `pytest` test.

Trap focus: bare `except:`; swallowing errors; `assert` ≠ validation. **Homework:** `ask_int` (EAFP validation loop); three more pytest cases; error triage.

Session 8 — Files, Libraries & Research Data

Objectives

1. Read/write text with `open` / `with` and understand file modes (why `"w"` is dangerous).
2. Load and write **CSV** survey/gradebook data with `csv.DictReader` / `DictWriter`; touch `json`.
3. `import` the researcher's stdlib (`statistics`, `datetime`, `pathlib`), `pip install` a package, and meet `pandas` in a guided teaser.

Trap focus: `"w"` silently overwrites; forgotten `newline=""`; reading a file twice; CSV values are strings. **Homework:** attendance-report pipeline; JSON round-trip; run the pipeline on your own CSV.

Session 9 — Regular Expressions & Text Cleaning

Objectives

1. Write patterns with raw strings and the survival tokens; know when *not* to use regex.
2. Use `re.search` / `fullmatch` / `findall` / `sub` to validate, extract (capture groups), and clean real research text.

Trap focus: forgetting `r"..."`; `.` matches any char; `re.search` returns `None` — guard before `.group()`. **Homework:** pattern drill (IDs, phones, dates); messy-name cleanup; domain harvest.

Session 10 — Modules, OOP & the Pythonic Toolkit

Objectives

1. Split code into modules and `import` them; understand the `if __name__ == "__main__":` guard.
2. Model a domain entity with a small **class** (`__init__`, `self`, `__str__`, a validating `@property`, brief inheritance with `super()`).
3. Apply the **Pythonic toolkit**: comprehensions, `map` / `filter`, generators/ `yield`, the walrus `:=`.

Trap focus: `self` confusion; a generator exhausts after one pass; the shared class variable; over-using a class where a function/dict fits. **Homework:** `Student` with computed GPA; a `Cohort` class; Pythonic rewrite of three loops.

Session 11 (Optional) — Capstone Project *(integrative)*

Objective: independently build one small, end-to-end program on a real-ish education dataset. Default brief: **"Gradebook & Survey Analyzer"** — read a CSV of students + Likert responses, clean and validate it, compute summary statistics, flag at-risk students, and write a report CSV. Alternative briefs are listed in `assessments/capstone-project.md`. Plan ~2 hours.

Coverage check (every core topic lands somewhere)

Topic	Where it lives now
Running Python, variables & types	S1
The dynamic-typing traps (`==` / `is` , floats, `bool⊂int` , `isinstance`)	S2
Conditionals & loops	S3
Data structures (list/tuple/dict/set)	S4
Functions, scope & reuse	S5
Recursion	S6 (nested data ties back to S4)
Exceptions & unit tests	S7
Files & libraries (CSV, `statistics` , `pandas` teaser)	S8
Regular expressions	S9
Modules & OOP	S10
Power-tools (comprehensions, `*args` , type hints, generators, `map` / `filter` , walrus)	S4, S5, S10

Scaling to the time budget

- **~8 hours:** merge S1+S2 and S3+S4 into two fast 90-minute sessions (he's quick) and trim the S10 toolkit tour and the S8 `pandas` / `json` teaser.
- **~10 hours:** run S1–S10 as written, one hour each (recommended).
- **~12 hours:** add the S11 capstone.

Appendix B — Connection Map (education-research bridges)

Connection Map — Python ⇄ Education-Research Background

> From the *personalized-syllabus* method: each new programming concept is bridged to something the student already knows from education research, and — crucially — **where the bridge breaks** is named, so the new concept isn't quietly collapsed into the familiar one. Use these as the "hooks" when introducing each topic; they cut learning time dramatically for an expert adult.

---

1. Variables & assignment *(S1)*

- **Maps onto:** named quantities in a stats model — `n`, `mean`, `alpha`.
- **Where it breaks:** in math, `x = x + 1` is a contradiction; in Python `=` is an *action* (rebind the name), not an assertion of equality. A variable is a **label stuck on an object**, not a box that holds a value. This single reframing prevents most aliasing confusion later.

2. Types & dynamic typing *(S1–S2)*

- **Maps onto:** measurement levels — nominal/ordinal/interval/ratio. Python's `str` / `bool` / `int` / `float` are loosely analogous (categorical vs counted vs continuous).
- **Where it breaks:** SPSS/R columns have *one declared type per column*; a Python variable can hold a string now and an int later. The discipline a column gives you for free, you must supply yourself. This is exactly why the comparison traps in S2 exist.

3. `==` vs `is` *(S2)*

- **Maps onto:** the difference between **two questionnaires with identical answers** (equal in value) and **the same physical respondent** (identical individual). Two students can have the same GPA (`==`) without being the same person (`is`).
- **Where it breaks:** the analogy is exact for objects, but Python adds an implementation wrinkle (small-integer caching) that has no analogue in research — flagged as a "do not rely on this."

4. Boolean as a subclass of int (`True == 1`) *(S2)*

- **Maps onto: dummy coding** — you already encode "treatment = 1, control = 0" and then *sum* the dummies to count cases. Python literally does this: `sum([True, False, True]) == 2`.

- **Where it breaks:** in your data the 1/0 is a convention you imposed; in Python it's baked into the language, so `True + True == 2` works even when you didn't intend arithmetic on a flag.

5. Float precision (`0.1 + 0.2 != 0.3`) *(S2)*

- **Maps onto: rounding/measurement error**. You already never test two measured scores for exact equality; you use tolerances and report to 2 decimals.
- **Where it breaks:** here the error isn't in the data — it's in how the *computer* stores decimals in binary. Same instinct (`math.isclose`, round for display), new cause.

6. Lists / dicts / DataFrames *(S4, S8)*

- **Maps onto:** a `list` of `dict`s is a **tidy dataset**: each dict is a *row/respondent*, each key is a *variable/column*. A `dict` alone is one record's variable→value map.
- **Where it breaks:** a spreadsheet enforces rectangularity; a list of dicts does not — rows can have missing or extra keys, which is the source of many bugs (and why we validate in S7).

7. Functions *(S5)*

- **Maps onto:** a **statistical formula or a coding scheme**: defined once, applied to many cases, same rule every time → reproducibility, the thing you care about most as a researcher.
- **Where it breaks:** functions can carry *hidden state* (mutable defaults, globals) so the "same input → same output" promise can silently fail. That trap (S5) is the whole reason to learn scope.

8. Exceptions & validation *(S7)*

- **Maps onto: data cleaning** — out-of-range Likert values, blank cells, "N/A" typed into a numeric field. You already have a mental model of dirty data; exceptions are how code reacts to it.
- **Where it breaks:** cleaning in SPSS is a one-time batch pass; in a program you decide *at runtime*, per value, whether to skip, fix, or stop — a more granular control than a recode syntax file.

9. Regular expressions *(S9)*

- **Maps onto: search-and-filter in a corpus / qualitative coding** — finding every response matching a pattern, extracting IDs, normalizing free-text.
- **Where it breaks:** regex matches *surface form*, not meaning. It's powerful for structure (emails, dates, IDs) and treacherous for semantics — the opposite trade-off from human coding.

10. Classes / OOP *(S10)*

- **Maps onto:** an **operational definition / construct** — a `Student` class bundles the attributes and behaviors you've decided "count" as a student, like an operationalized variable.

- **Where it breaks:** a construct is a measurement choice; a class is also *behavior* (methods) and *enforced rules* (validation in setters), so it's a construct that can defend its own integrity.

11. Recursion *(S6)*

- **Maps onto:** a **hierarchy / nested structure** — coding schemes with sub-codes, threaded discussion data, folder trees of data files, nested JSON survey exports. A procedure "defined in terms of itself" mirrors data that contains smaller copies of itself.
- **Where it breaks:** recursion is elegant only when the structure is genuinely nested and the base case is reachable; for a flat sequence, a loop is clearer, and Python's stack limit (~1000, no tail-call optimization) makes very deep recursion a liability rather than a virtue.

Questions to hold while learning (Socratic spine)

These recur across sessions; the teacher should keep returning to them.

1. *Is this a question about value, or about identity?* (drives `==` vs `is`, copy vs alias)
2. *What type is this right now, and who decided that?* (dynamic typing discipline)
3. *Could this input be dirty? What happens if it is?* (defensive mindset)
4. *Is the same rule guaranteed to run the same way every time?* (reproducibility / pure functions)
5. *Is there a more Pythonic, more readable way to say this?* (the Zen of Python's "readability counts")

Appendix C — Per-Session Quizzes & Answer Keys

Per-Session Quizzes (with answer keys)

Each quiz is 4–5 questions, ~6 minutes, given at the end of the session. Every question maps to a session objective. **Teacher:** ask the student to *predict* code output before they run it — the surprise is the assessment. Answers are at the end of each session block.

Session 1 — Running Python, Variables & Types

1. What type does `input("x: ")` return, always?
2. What does `"5" + "3"` evaluate to? How do you get `8`? And what is `int(3.9)`?
3. What does `f"{0.873:.1%}"` produce?
4. In a traceback, which line do you read first — and what two things does it tell you?

Answers: 1. `str`. 2. `"53"`; use `int("5") + int("3")`; `int(3.9)` is `3` (truncates, doesn't round). 3. `"87.3%"`. 4. The **last** line: the exception's name (what kind of problem) and the message (the offending value or detail).

Session 2 — The Dynamic-Typing Traps

1. `a = [1,2]; b = [1,2]` — is `a == b`? Is `a is b`? Why?
2. What is `True + True`? Why?
3. Is `0.1 + 0.2 == 0.3` True or False? How *should* you compare them?
4. What does `5 == "5"` give? What about `5 > "5"`?
5. Which of these are falsy: `"0"`, `""`, `[0]`, `[]`, `None`, `0.0`?

Answers: 1. `==` True (same value), `is` False (different objects in memory).

2. `2` — bool is a subclass of int (`True` acts as `1`). 3. False (binary float rounding); use `math.isclose(0.1 + 0.2, 0.3)` or round. 4. `False` (no error); `5 > "5"` raises `TypeError` — Python can't *order* int vs str. 5. `""`, `[]`, `None`, `0.0` are falsy; `"0"` and `[0]` are truthy (non-empty).

Session 3 — Control Flow: Conditionals & Loops

1. Rewrite `if x >= 90 and x < 100:` using a chained comparison.
2. What is `5 and 0`? What is `0 or "hi"`? Why aren't they `True` / `False`?
3. What does `list(range(1, 5))` produce?
4. What goes wrong when you `remove` items from a list while looping over it — and what's the fix?

Answers: 1. `if 90 <= x < 100:`. 2. `0` and `"hi"` — `and` / `or` return an *operand*, not a bool. 3. `[1, 2, 3, 4]` (5 excluded — off-by-one). 4. Removal shifts the remaining indices, so elements get skipped; build a new list instead (`[x for x in xs if keep(x)]`) or iterate over a copy.

Session 4 — Data Structures

1. Which are mutable: list, tuple, dict, set?
2. `a = [1,2]; b = a; a.append(3)` — what is `b`? Why?
3. (Practical) Write a one-line dict comprehension mapping each name in `names` to `0`.
4. `xs.sort()` vs `sorted(xs)` — what does each return?
5. You need "have I seen this ID before?" over thousands of IDs — which structure, and why?

Answers: 1. list, dict, set are mutable; tuple is not. 2. `[1, 2, 3]` — `b` is an alias for the *same* list (assignment doesn't copy). 3. `{n: 0 for n in names}`.

4. `.sort()` sorts in place and returns `None`; `sorted()` returns a *new* sorted list.
5. A `set` — membership tests are fast and duplicates are impossible by construction.

Session 5 — Functions, Scope & Reusability

1. What's the difference between `return` and `print`?
2. Why is `def f(x, items=[])` dangerous? What's the fix?
3. What does a function with no `return` statement return? Are type hints enforced at runtime?
4. `count = 0` at top level, then inside a function `count = count + 1` — what error, and why?

Answers: 1. `return` hands a value to the caller; `print` only displays it.

2. The default list is created once, at `def` time, and persists across calls; use `items=None` then create inside. 3. `None`; and no — hints are documentation (`mypy` checks them optionally). 4. `UnboundLocalError` — the assignment makes `count` local to the function, so the right-hand side reads a local that doesn't exist yet.

Session 6 — Recursion & Recursive Thinking

1. What two parts must every recursive function have?
2. What error comes from a base case that's never reached, and why doesn't Python just keep going?
3. Why is recursion a natural fit for nested data (a list of lists, or nested JSON)?
4. The recursive case computes `n * f(n - 1)` but has no `return` in front — what does a call produce?

Answers: 1. A **base case** (stops) and a **recursive case** (calls itself on a smaller input, *toward* the base case). 2. `RecursionError` — Python has no tail-call optimization, so each pending call keeps a stack frame until the limit (~1000).

3. The data is *defined in terms of itself* (a list may contain lists), so a function defined in terms of itself mirrors its shape and reaches every level. 4. `None` — the value is computed and thrown away; the function falls off the end.

Session 7 — Exceptions & Defensive Code

1. Which exception does `int("N/A")` raise? Wrap `int(value)` so it returns `None` on failure.
2. Why is a bare `except:` dangerous?
3. When should you use `raise` vs `assert`?
4. Name the two validation styles — and give the EAFP version of converting `value` to an int.

Answers: 1. `ValueError`; `try: return int(value)` / `except (ValueError, TypeError): return None`.

2. It catches *everything* (even Ctrl+C and your own typos) and can hide bugs.
3. `raise` to validate real/untrusted input; `assert` for developer sanity checks (it can be disabled with `python -O`). 4. LBYL ("look before you leap": `if value.isdigit():`) vs EAFP ("easier to ask forgiveness"): `try: n = int(value)` / `except ValueError: ...`.

Session 8 — Files, Libraries & Research Data

1. What does opening a file in `"w"` mode do to existing contents? Why prefer `with open(...)`?
2. After `csv.DictReader`, what type is each row?
3. CSV values read from a file are what type — and what must you do with numbers?
4. Your second loop over an open file object sees nothing — why, and what's the fix?

Answers: 1. Truncates it to empty *immediately*; `with` auto-closes the file even if the code crashes. 2. A `dict` keyed by the header row. 3. Strings; convert with `int()` / `float()`. 4. The file cursor is exhausted after one pass — re-open the file (or read it into a list first).

Session 9 — Regular Expressions & Text Cleaning

1. In regex, what does `.` match? How do you match a literal dot?
2. Why write regex patterns as raw strings `r"..."`?

3. What does `re.search` return when there's no match, and what must you do before `.group()`?

4. `re.search` vs `re.fullmatch` — which one is for validation, and why?

Answers: 1. Any character (except newline); use `\.` for a literal dot. 2. So backslashes aren't treated as Python string escapes. 3. It returns `None`; check `if m:` before `m.group()` or you'll hit an `AttributeError`. 4. `fullmatch` — it anchors both ends, so the *whole* string must fit the pattern; `search` would accept a valid-looking fragment inside garbage.

Session 10 — Modules, OOP & the Pythonic Toolkit

1. In a class, what is `self`?
2. What happens if you iterate a generator twice?
3. Why doesn't a module's `if __name__ == "__main__":` block run when you `import` it?
4. `class Course: students = []` — what goes wrong when two `Course` objects enroll students, and what's the fix?

Answers: 1. The current instance ("this particular object"). 2. The second pass is empty — a generator is exhausted after one iteration. 3. On import, `__name__` is the module's name, not `"__main__"`, so the block is skipped. 4. `students` is a *class* variable shared by every instance, so both courses see one combined list; give each its own in `__init__`: `self.students = []`.

Scoring guide (formative, not graded)

- **All correct, explained why:** ready for the next session (or the capstone).
- **Right answer, fuzzy why:** re-do that session's `traps-and-gotchas` rows.
- **Wrong on Session 2 items:** revisit the type traps before continuing — they're load-bearing for everything after.